

Less Is Personal: Learning Minimal Sufficient User Profiles for Personalized Language Models

Minghang Liu^{♣,♡}, Qiang Qiu[♣], Yuanzhuo Wang[♣]
Huawei Shen[♣], Xueqi Cheng[♣]

[♣]State Key Laboratory of AI Safety, Institute of Computing Technology, CAS

[♡]University of Chinese Academy of Sciences, Beijing, China

{liuminghang23s, wangyuanzhuo, qiangqiu, shenhuawei}@ict.ac.cn

Abstract

Retrieval-augmented personalization enables large language models to produce more accurate and preference-aligned outputs using relevant records retrieved from user histories. Personalized language models typically prepend a fixed number of retrieved user records, even when additional history is redundant, harmful, or unrelated to a user’s distinctive behavior. We study minimal sufficient personalization: constructing the least costly ordered profile for each input while preserving the utility achievable from a retrieved candidate pool. We introduce ENOUGH, a method that iteratively appends behavioral records or emits STOP to construct profiles with adaptive lengths. Offline, bounded counterfactual search evaluates profile prefixes by jointly considering downstream gains, user specificity, and token costs. The resulting long-horizon targets are distilled into a multi-head value controller with explicit ranking and stopping supervision. At inference, the controller selects and orders records through lightweight decisions, and the frozen generator is invoked once after stopping. Extensive experiments on six personalized tasks demonstrate that ENOUGH consistently outperforms strong heuristic and retrieval-augmented baselines in both effectiveness and efficiency, achieving minimal sufficient profiles that preserve personalization utility while reducing unnecessary context costs.

1 Introduction

Personalized large language models (LLMs) adapt predictions and generations to a user’s preferences, expertise, and behavioral patterns, supporting tasks from classification and recommendation to tailored text generation (Salemi et al. 2024a; Zhang et al. 2025b). One approach encodes the user in model parameters or a static textual profile (Zhang 2024; Tan et al. 2024). Such persistent representations, however, can be costly to update or may compress away the fine-grained, request-dependent evidence in a growing interaction history. Retrieval-augmented personalized LLMs offer a flexible approach: for each request, they retrieve a set of the user’s past behavioral records and place them in the prompt, allowing a frozen generator to adapt without per-user retraining (Salemi et al. 2024a; Xu et al. 2025).

However, making history accessible does not determine how it should be used. A typical retrieval-augmented pipeline ranks records by semantic relevance and prepends a fixed top- k profile. Recent methods improve this pipeline through

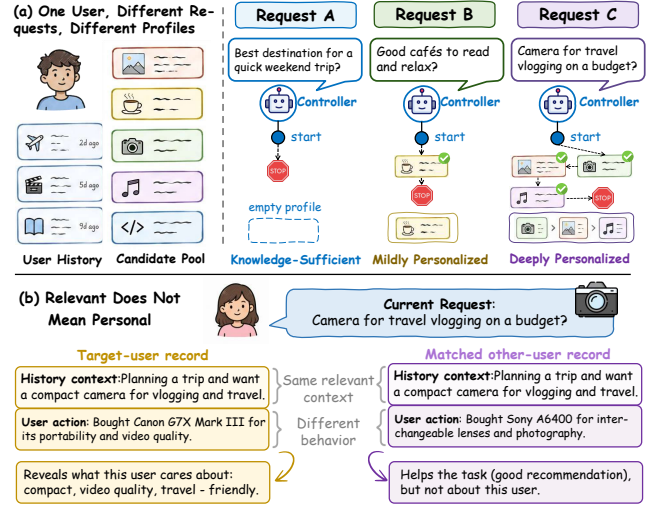


Figure 1: (a) A user’s requests may require profiles of different lengths, including an empty one. (b) Query-relevant records can encode different user actions, so usefulness need not imply target-user specificity.

generation feedback (Salemi et al. 2024b), attention-derived relevance scoring (Chen et al. 2025), or order-aware profile policies (Du et al. 2026), but the resulting profile is still commonly a fixed-size set or fixed-length permutation. This view leaves two questions unresolved: *how much history should be used?* and *which useful history is genuinely personal?*

How much history should be used? The useful amount of history depends on both the current request and the records already selected. As illustrated in Figure 1(a), a request may be answerable without personal context, may need a single revealing behavior, or may require several selected records. A fixed profile length offers no mechanism to recognize when later records merely repeat an established preference, introduce conflicting signals, or dilute useful context. Personalized generation should thus be viewed as a selective, self-terminating evidence-acquisition process, not an always-on retrieval regime. Model should decide whether history is needed at all and identify when further history ceases.

Which useful history is genuinely personal? Relevance,

general task utility, and personalization answer different questions. Semantic similarity indicates whether a historical context concerns the current request, but not whether the associated action reveals the target user’s behavior. Likewise, a record can improve generation simply by serving as a generally useful in-context example. Figure 1(b) illustrates this distinction: target-user and other-user records share the same relevant context, yet their actions imply different preferences. We operationalize personal value through two contrasts. The no-profile contrast asks whether the complete selected profile helps at all. For a beneficial profile, content-matched other-user replacements then ask whether a target-user record contributes more than a comparably relevant record while the remaining profile is held fixed. Thus, for our frozen generator, history is valued neither by semantic similarity nor by generic generation gain, but by its *user-specific marginal value* under the current profile.

These two questions are inseparable. The marginal value of a candidate changes with the selected profile: a record that is useful initially can become redundant after a similar behavior has been included, while records may interact or complement one another. Consequently, history length should not be a hyperparameter chosen independently of record value. It should emerge from the same conditional decision process: continue while some candidate offers future personalized value, and stop when the current profile is sufficient. We call this objective *minimal sufficient personalization*: within a retrieved candidate pool, seek the least costly ordered profile that approaches its best attainable personalized utility.

We propose Efficient Net-utility Optimization for User-profile Growth and Halting (ENOUGH), a method that targets this objective by either appending a behavioral record or emitting STOP. Starting from the empty profile, ENOUGH can reject profile augmentation at the root, choose and order interacting records, and terminate at any feasible length. Offline, bounded counterfactual search evaluates profile prefixes through three quantities: downstream gain over no personalization, beneficial user specificity measured against content-matched other user replacements, and actual incremental prompt-token cost. Long-horizon backups then supervise a multi-head value controller, so selection and stopping account for future record interactions rather than only immediate gains. At inference, the controller constructs the profile through lightweight decisions, and the frozen generator is invoked once after stopping. Our contributions are threefold:

- We formulate retrieval-augmented personalization as minimal sufficient profile construction, unifying whether to personalize, which behavioral records to select, and when additional history is no longer worthwhile.
- We introduce a dual counterfactual utility that separates general downstream gain from user-specific marginal value, and a long-horizon sequential controller with an explicit STOP action.
- Our evaluation spans six tasks across a broad range of personalization settings, including classification, regression, and text generation, together with diagnostics of stopping, specificity, robustness, and efficiency.

2 Methodology

As shown in Figure 2, ENOUGH constructs an input-adaptive ordered user profile that targets minimal sufficiency from the top- M candidates of a frozen retriever. Starting from the empty profile, a lightweight controller sequentially appends a behavioral record consisting of its historical context and the user’s action or selects STOP. Offline, bounded counterfactual search evaluates profiles using downstream gain, target-user specificity, and token cost, then distills long-horizon action values into the controller. At test time, construction uses neither references nor generator rollouts; the frozen generator is invoked once after stopping.

2.1 Problem Formulation

Let the training set be

$$\mathcal{D} = \{(u_i, \tau_i, x_i, y_i, H_{u_i})\}_{i=1}^N, \quad (1)$$

where u_i denotes a profile owner, τ_i its personalization task, x_i the current input, y_i its reference output, and H_{u_i} the history. Each history item is $d = (c_d, a_d, m_d)$, with c_d is the historical context, a_d is an observed user action (e.g., a label, score, or generated text), and m_d contains optional metadata such as timestamps. A task-specific serializer ϕ_τ maps each record into a common textual schema.

A frozen retriever R constructs a candidate pool

$$C_i = R(x_i, H_{u_i}; M), \quad |C_i| \leq M, \quad (2)$$

where M is the candidate-pool size limit and we set $K_i = \min(K_{\max}, |C_i|)$.

We consider the space $\mathfrak{S}_i$ of ordered, non-repeating sequences from C_i whose length is at most K_i and whose rendered prompt satisfies a hard context limit $L_{\max}$. At step t the controller holds $\mathbf{S}_t = (d_1, \dots, d_t)$ and chooses

$$a_t \in \mathcal{A}(s_t) = \{d \in C_i \setminus \mathbf{S}_t : \mathbf{S}_t \oplus d \in \mathfrak{S}_i\} \cup \{\text{STOP}\}. \quad (3)$$

Selecting d appends it to the ordered profile; selecting STOP terminates construction. The final output is

$$\hat{y}_i = G(P_{\tau_i}(x_i, \mathbf{S}_T)), \quad (4)$$

where G is the frozen generator and P_τ is prompt template.

Minimal sufficient personalization. Let $U_i(\mathbf{S})$ denote the cost-free personalized utility defined in Section 2.3, and let

$$U_i^* = \max_{\mathbf{S} \in \mathfrak{S}_i} U_i(\mathbf{S}). \quad (5)$$

An ϵ -minimal sufficient profile is

$$\mathbf{S}_i^{\text{MSP}} \in \arg \min_{\mathbf{S} \in \mathfrak{S}_i} c_i(\mathbf{S}) \quad \text{s.t.} \quad U_i(\mathbf{S}) \geq U_i^* - \epsilon. \quad (6)$$

This definition separates sufficiency from minimality. Because the ordered search space is discrete and non-convex, we optimize the cost-regularized surrogate:

$$J_i^\lambda(\mathbf{S}) = U_i(\mathbf{S}) - \lambda c_i(\mathbf{S}). \quad (7)$$

Different values of λ recover supported points on the empirical quality–cost frontier.

We measure the actual incremental prompt cost:

$$c_i(\mathbf{S}) = \frac{\text{Tok}(P_{\tau_i}(x_i, \mathbf{S})) - \text{Tok}(P_{\tau_i}(x_i, \emptyset))}{L_{\text{ref}}}, \quad (8)$$

where L_{ref} is a profile reference budget shared by all tasks.

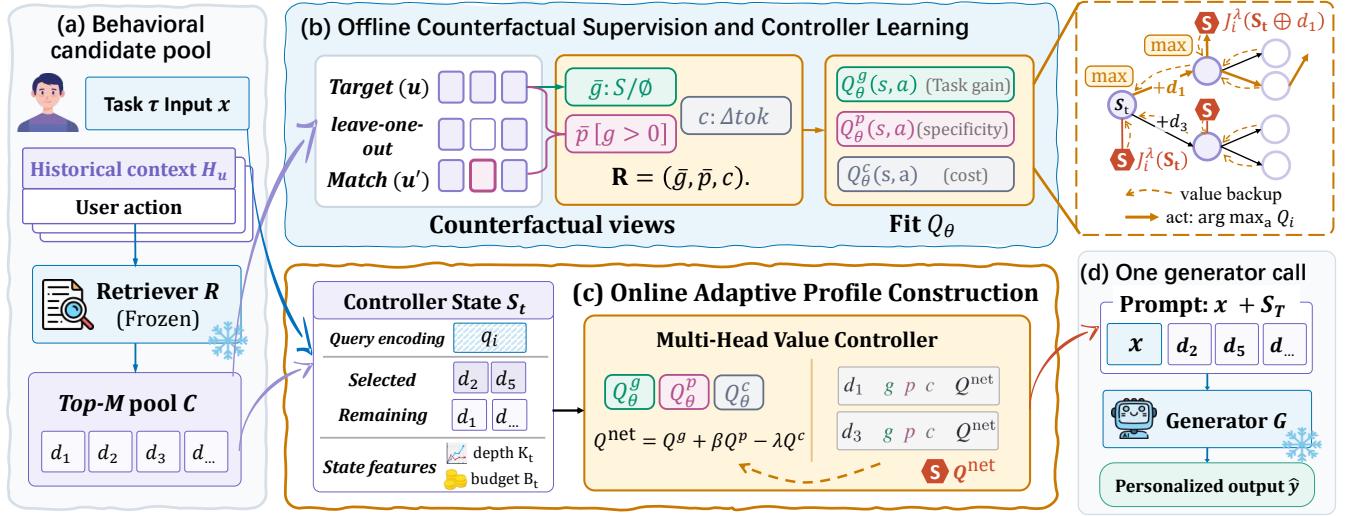


Figure 2: Overview of the ENOUGH framework. (a) A frozen retriever forms a behavioral candidate pool. (b) Offline counterfactual views measure downstream gain, matched-replacement specificity, and prompt cost; value backup supplies aligned long-horizon targets for the controller. (c) At test time, the controller adaptively scores feasible record additions and Stop by net value. (d) Once construction stops, the ordered profile and current input are passed to the frozen generator in one call.

2.2 Behavior-Aware Profile Representation

Personalization histories are observations of user behavior. The controller therefore encodes both the situation and the user’s response. For a candidate record d_j , we compute

$$e_j = E_h([\text{TASK} = \tau_i] \parallel [\text{HIST_CONTEXT}] \parallel c_{d_j} \parallel [\text{USER_ACTION}] \parallel a_{d_j} \parallel [\text{META}] \parallel m_{d_j}). \quad (9)$$

The current request is encoded as $q_i = E_q([\text{TASK} = \tau_i] \parallel [\text{QUERY}] \parallel x_i)$.

The controller state induced by the ordered profile S_t is

$$s_t = (q_i, z_t, r_t, b_t), \quad (10)$$

where $z_t = E_S(e_{d_1}, \dots, e_{d_t})$ is an order-sensitive incremental representation of the selected profile. $r_t = \text{AttnPool}([q_i; z_t], \{e_j : d_j \in C_i \setminus S_t\})$ is a permutation-invariant summary of the remaining candidates. b_t contains the current incremental token cost, remaining horizon, and remaining context budget. In the multi-task setting, the encoders and value controller are shared across personalization tasks, while the task embedding, serializer, and prompt template are task-conditioned.

2.3 Dual Counterfactual Profile Utility

Downstream task gain For a user profile S , we compute the length-normalized log-likelihood of the reference output:

$$\ell_i(S) = \frac{1}{|y_i|} \sum_{r=1}^{|y_i|} \log p_G(y_{i,r} \mid y_{i,<r}, P_{\tau_i}(x_i, S)). \quad (11)$$

The downstream gain over no profile is

$$g_i(S) = \ell_i(S) - \ell_i(\emptyset). \quad (12)$$

To prevent high variance tasks from dominating shared training, we use a scale-only normalization. On a fixed calibration subset of the initial offline branches, we compute

$$s_\tau^g = \sqrt{\mathbb{E}_{i:\tau_i=\tau} [g_i(S)^2]}, \quad \bar{g}_i(S) = \frac{g_i(S)}{s_{\tau_i}^g + \varepsilon}. \quad (13)$$

We do not subtract a mean, preserving $\bar{g}_i(\emptyset) = 0$.

User-specific gain over matched replacements A profile can improve the likelihood simply because it contains a useful in-context demonstration. To isolate user-specific usefulness, we intervene on one profile slot while holding the rest of the selected profile fixed.

For every candidate d owned by u_i , we precompute K^- matched replacements $m_k(d; u_i)$ from other profile owners. Matching uses only variables available before observing the user action: task identity, historical context semantics, relevance to the current query, length, and time when present.

For a profile $S = (d_1, \dots, d_t)$ and position r , let S_{-r} be the sequence with d_r removed while preserving the order of all other records. For each slot, the target-user and matched-control marginal gains are then

$$\Delta_{i,r}^u(S) = \ell_i(S) - \ell_i(S_{-r}), \quad (14)$$

$$\Delta_{i,r}^{c,k}(S) = \ell_i(S_{-r} \oplus m_k(d_r; u_i)) - \ell_i(S_{-r}). \quad (15)$$

The slot-level beneficial specificity is

$$\rho_{i,r}(S) = [\Delta_{i,r}^u(S)]_+ - \frac{1}{K^-} \sum_{k=1}^{K^-} [\Delta_{i,r}^{c,k}(S)]_+. \quad (16)$$

We credit specificity only when the complete target-user profile improves over the empty profile:

$$p_i(S) = \mathbb{I}[g_i(S) > 0] \frac{1}{t} \sum_{r=1}^t \rho_{i,r}(S), \quad p_i(\emptyset) = 0. \quad (17)$$

This prevents a profile that is harmful relative to no personalization from receiving a positive specificity bonus merely because the matched controls are even worse. A single uniformly sampled position per profile yields an unbiased Monte Carlo estimator of Eq. (17).

As with task gain, we compute a scale on the fixed calibration subset and freeze it:

$$s_\tau^p = \sqrt{\mathbb{E}_{i:\tau_i=\tau} [p_i(S)^2]}, \quad \bar{p}_i(S) = \frac{p_i(S)}{s_{\tau_i}^p + \varepsilon}. \quad (18)$$

The cost-free personalized utility objective is

$$U_i(\mathbf{S}) = \bar{g}_i(\mathbf{S}) + \beta \bar{p}_i(\mathbf{S}). \quad (19)$$

Here, $\bar{g}_i(\mathbf{S})$ rewards profiles that improve task likelihood over the empty profile, while $\bar{p}_i(\mathbf{S})$ measures the additional benefit of target-user records over matched records from other users, discounting generic few-shot effects. The coefficients β and λ balance user specificity and prompt cost, respectively.

2.4 Long-Horizon Decision Principle

For a candidate pool, context budget and maximum depth, the theoretical optimal value obeys the finite-horizon recursion

$$V_i^*(\mathbf{S}) = \max \left\{ J_i^\lambda(\mathbf{S}), \max_{d \in \mathcal{A}(\mathbf{S}) \setminus \{\text{STOP}\}} V_i^*(\mathbf{S} \oplus d) \right\}. \quad (20)$$

Similarly,

$$Q_i^*(s, \text{STOP}) = J_i^\lambda(\mathbf{S}), Q_i^*(s, d) = V_i^*(\mathbf{S} \oplus d). \quad (21)$$

Equation (20) is a Bellman characterization, not a claim that the learned controller recovers the full combinatorial optimum. A detailed proof is provided in Appendix D.1.

Proposition 1 (Exact-value stopping). *On the finite deterministic candidate tree, if $Q_i^*(s, a)$ is known for every reachable state and action, selecting an action with maximum Q_i^* at each step, with ties resolved in favor of STOP, returns a terminal profile that maximizes J_i^λ within the candidate tree.*

2.5 Offline Counterfactual Supervision and Controller Learning

The complete ordered candidate tree is intractable. We construct a bounded prefix tree $\mathcal{T}_i$ using offline counterfactual search and perform exact backups only within that tree. The resulting target $\hat{Q}_{\mathcal{T}_i}$ is exact for the sampled tree but is only an approximation to Q_i^* .

Bounded-tree construction. We first sample tree roots from the empty profile, prefixes of the frozen retriever, recency and diversity policies, and random subsets. Profile sizes are stratified from zero to K_i so that the controller observes both early and late stopping states.

At each sampled root, STOP is always evaluated, and all feasible remaining candidate actions are expanded. At deeper states, a fixed action budget J combines high retriever-score, diversity, and uniformly sampled actions. Search retains at most W prefixes per depth. Each intermediate prefix is a legal terminal node. Unique ordered prefixes are represented once in directed acyclic graph structured cache, and all teacher-forced generator evaluations are batched.

Long-horizon target backup. For an expanded state-action pair (s, a) , let $\mathcal{L}_{\mathcal{T}}(s, a)$ denote the terminal nodes in the sampled subtree reached after taking a . We select one terminal node using the same composite objective for all three value components:

$$L_{\mathcal{T}}^*(s, a) = \arg \max_{L \in \mathcal{L}_{\mathcal{T}}(s, a)} J_i^\lambda(L). \quad (22)$$

The vector-valued target is then $\mathbf{R}_i(s, a) = (R_i^g, R_i^p, R_i^c)$.

For STOP, the leaf is the current profile. Thus, the three components always describe the same optimal continuation within the sampled tree. Define the corresponding scalar target as $R_i^{\text{net}}(s, a) = R_i^g(s, a) + \beta R_i^p(s, a) - \lambda R_i^c(s, a)$.

Multi-head value controller. For every available action, the controller predicts three outcome components of the net-optimal sampled continuation: $Q_\theta^g(s, a)$, $Q_\theta^p(s, a)$, and $Q_\theta^c(s, a)$. These quantities estimate normalized task gain, normalized matched-replacement specificity, and normalized prompt cost, respectively. The composite score is

$$Q_\theta^{\text{net}}(s, a) = Q_\theta^g(s, a) + \beta Q_\theta^p(s, a) - \lambda Q_\theta^c(s, a). \quad (23)$$

For any state with at least one feasible record action, define the predicted stop margin

$$\Delta_\theta^{\text{stop}}(s) = Q_\theta^{\text{net}}(s, \text{STOP}) - \max_{d \in \mathcal{A}(s) \setminus \{\text{STOP}\}} Q_\theta^{\text{net}}(s, d). \quad (24)$$

Controller optimization. Let $\mathcal{A}_{\text{lab}}(s)$ be the action subset expanded and labeled in the bounded tree. For each labeled state-action pair, we regress the three outcome components:

$$\mathcal{L}_{\text{value}} = \mathbb{E}_{(s, a)} \sum_{k \in \{g, p, c\}} w_k \text{Huber} \left(Q_\theta^k(s, a), R_i^k(s, a) \right). \quad (25)$$

In addition to pointwise value regression, we encourage the controller to preserve the relative preferences among the labeled actions at each state. The bounded-tree target and controller distributions over the same action subset are

$$p^*(a | s) = \text{softmax}_{a \in \mathcal{A}_{\text{lab}}(s)} (R_i^{\text{net}}(s, a) / T_r), \quad (26)$$

$$p_\theta(a | s) = \text{softmax}_{a \in \mathcal{A}_{\text{lab}}(s)} (Q_\theta^{\text{net}}(s, a) / T_r). \quad (27)$$

The resulting listwise ranking loss is

$$\mathcal{L}_{\text{rank}} = \mathbb{E}_s [D_{\text{KL}}(p^*(\cdot | s) \| p_\theta(\cdot | s))]. \quad (28)$$

The stop-decision loss is applied only to fully expanded states with at least one record action, for which the labeled action mask contains every feasible action. This restriction prevents an unexpanded beneficial action from creating a false positive STOP label. For such a state, define the bounded-tree teacher margin as

$$\Delta_{\mathcal{T}_i}^{\text{stop}}(s) = R_i^{\text{net}}(s, \text{STOP}) - \max_{d \in \mathcal{A}(s) \setminus \{\text{STOP}\}} R_i^{\text{net}}(s, d). \quad (29)$$

Using the soft target $y_{\text{stop}} = \sigma(\Delta_{\mathcal{T}_i}^{\text{stop}} / T_s)$, we set

$$\mathcal{L}_{\text{stop}} = \text{BCE}(\sigma(\Delta_{\mathcal{T}_i}^{\text{stop}} / T_s), y_{\text{stop}}). \quad (30)$$

The complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{value}} + \eta \mathcal{L}_{\text{rank}} + \zeta \mathcal{L}_{\text{stop}}. \quad (31)$$

Mini-batches are task-balanced by uniform task sampling.

2.6 Online Adaptive Profile Construction

After offline training, only the learned controller parameters cross the offline-online boundary. Matched replacements, reference likelihoods, and counterfactual search are discarded. The deployment retains only R , the controller (with its training-time β, λ), and the frozen generator G .

The learned policy replaces the unavailable oracle value with the predicted composite score:

$$\pi_\theta(s) = \arg \max_{a \in \mathcal{A}(s)} Q_\theta^{\text{net}}(s, a), \quad (32)$$

with ties resolved in favor of STOP. When at least one record action remains feasible, the controller stops if and only if the predicted margin $\Delta_\theta^{\text{stop}}(s)$ in Eq. (24) is non-negative. A STOP decision at the root yields no profile augmentation, although candidate retrieval has still been performed.

Task	Metric	BM25	Contriever	IC-RALM	REPLUG	RankGPT	ICR	RSPG-Pre	PURPLE	ENOUGH
<i>With Qwen3.5-9B</i>										
LaMP-1	Acc./F1	63.1/61.5	64.5/63.0	<u>67.1/65.6</u>	65.7/64.1	67.0/65.3	64.8/63.4	66.2/64.7	66.7/65.1	68.6*/67.0*
LaMP-2	Acc./F1	50.2/45.8	49.8/44.9	50.1/46.0	48.7/44.7	50.2/46.0	51.2/46.6	50.3/46.1	<u>52.1/47.0</u>	53.9*/48.6*
LaMP-3	MAE/RMSE	31.7/68.8	31.4/67.6	29.3/66.2	28.5/62.4	<u>28.2/61.9</u>	30.5/67.0	29.4/66.4	29.0/65.5	26.9*/58.3*
LaMP-4	R-1/R-L	19.0/16.7	18.8/17.0	17.7/16.8	19.1/17.5	<u>18.3/17.6</u>	19.1/17.3	18.2/17.2	<u>19.4/17.8</u>	19.9*/18.1*
LaMP-5	R-1/R-L	45.5/39.9	46.1/40.7	45.8/42.3	45.7/41.8	<u>47.5/42.8</u>	46.4/41.8	46.1/41.3	45.3/42.5	48.2*/43.6*
LaMP-7	R-1/R-L	49.3/44.8	48.8/45.6	51.7/47.0	51.0/47.5	49.8/46.9	49.8/47.1	49.4/46.1	<u>51.9/47.8</u>	53.0*/48.7*
<i>With Llama-3.1-8B-Instruct</i>										
LaMP-1	Acc./F1	62.2/59.7	63.7/61.4	64.0/62.5	64.8/62.1	<u>66.1/63.9</u>	63.9/61.1	65.3/62.7	65.8/63.4	67.6*/64.9*
LaMP-2	Acc./F1	47.9/43.8	48.9/44.0	49.2/45.1	49.4/44.9	<u>49.3/45.0</u>	50.3/45.7	49.5/45.2	<u>51.2/46.1</u>	52.9*/47.7*
LaMP-3	MAE/RMSE	32.6/70.0	32.3/67.7	30.3/68.6	29.4/67.5	30.2/65.1	31.4/64.1	<u>29.1/62.3</u>	29.9/68.0	27.8*/59.4*
LaMP-4	R-1/R-L	17.9/17.1	17.1/16.6	17.3/16.4	18.6/17.0	18.5/16.3	18.6/16.9	17.8/16.8	<u>18.9/17.3</u>	19.4*/17.6*
LaMP-5	R-1/R-L	44.7/39.2	45.3/39.9	45.0/41.5	44.9/41.0	45.6/41.5	<u>46.7/42.0</u>	45.3/40.5	44.5/41.7	47.4*/42.8*
LaMP-7	R-1/R-L	47.9/43.5	47.4/44.3	50.3/45.7	49.6/46.2	48.4/45.6	48.4/45.8	48.0/44.8	<u>50.5/46.5</u>	51.6*/47.4*

Table 1: Main results on the LaMP time-based split with Qwen3.5-9B (top) and Llama-3.1-8B-Instruct (bottom) as frozen task generators. Accuracy, F1, and ROUGE are reported on a 0–100 scale; MAE and RMSE are multiplied by 100. The best and second-best results within each generator are shown in bold and underlined, respectively. The superscript * denotes a statistically significant improvement over the strongest baseline under a two-sided paired t -test across the same ten runs ($p < 0.05$).

3 Experiments

3.1 Experimental Setup

Dataset and Evaluation. Our experiments use the **LaMP** benchmark (Salemi et al. 2024a), which covers six personalized tasks, including three classification tasks—**LaMP-1** (Citation Identification, binary), **LaMP-2** (Movie Tagging, multiclass with 15 categories), and **LaMP-3** (Product Rating, ordinal from 1–5 stars) and four text generation tasks: **LaMP-4** (News Headline Generation), **LaMP-5** (Scholarly Title Generation), and **LaMP-7** (Tweet Paraphrasing).

Following previous work (Salemi et al., 2024b,a; Shi et al., 2025), we report accuracy (Acc.) and F1 score for LaMP-1 and LaMP-2, mean absolute error (MAE) and root mean squared error (RMSE) for LaMP-3. We evaluate text generation performance on LaMP-4, LaMP-5, and LaMP-7 using ROUGE-1 (R-1) and ROUGE-L (R-L) (Lin, 2004). Dataset statistics, the time-based (T) split protocols, and detailed metric definitions are reported in Appendix E.1.

Implementation Details. We use Qwen3.5-9B (Qwen Team 2026) as the primary frozen task generator in all main experiments. We repeat the evaluation with Llama-3.1-8B-Instruct (Grattafiori et al. 2024) as an alternative frozen task generator. Our frozen retriever is Contriever (Izacard et al. 2022), which returns at most $M = 20$ legal records. Further implementation details are provided in Appendix E.2.

Baselines The main comparison contains eight retrieval-augmented baselines. BM25 (Robertson and Zaragoza 2009) and Contriever (Izacard et al. 2022) provide sparse and frozen dense relevance rankings. RankGPT (Sun et al. 2023) performs zero-shot listwise permutation reranking, whereas ICR (Chen et al. 2025) derives calibrated relevance scores from query-to-record attention. IC-RALM (Ram et al. 2023) incorporates records through decoding-time context switching, while REPLUG (Shi et al. 2024) marginalizes token predictions from record-conditioned generation streams using normalized retrieval scores. RSPG-Pre (Salemi et al.

2024b) uses a Longformer selector before generation to choose among multiple retrievers. PURPLE (Du et al. 2026) learns an order-sensitive Plackett–Luce profile policy using the frozen generator’s reference-response log-likelihood as reward. Detailed baseline configurations, shared budgets and prompts are provided in Appendix E.2.

3.2 Main Results

ENOUGH consistently achieves the best performance across tasks, metrics, and generator backbones. As shown in Table 1, it ranks first in all 24 reported metric cells: six tasks with two metrics each under both Qwen3.5-9B and Llama-3.1-8B-Instruct. With Qwen3.5-9B, ENOUGH improves over the strongest baseline by 1.5/1.4 points on LaMP-1 and by 1.8/1.6 points on LaMP-2. On the ordinal-rating task LaMP-3, it reduces MAE and RMSE by 1.3 and 3.6 points, respectively. The gains extend to all generation tasks: ENOUGH improves ROUGE-1/ROUGE-L by 0.5/0.3 points on LaMP-4, 0.7/0.8 points on LaMP-5, and 1.1/0.9 points on LaMP-7. Together, these results establish the effectiveness of ENOUGH across classification, ordinal prediction, and personalized generation. The same pattern transfers to Llama-3.1-8B-Instruct. The consistent improvements across two frozen generators support the robustness of ENOUGH to the choice of task backbone.

3.3 Ablation Analysis

Figure 3 compares four single-component ablations. *w/o Long-horizon* replaces long-horizon supervision with myopic targets, *w/o Adaptive Stop* forces the development-selected k , *w/o User-Specificity* sets $\beta = 0$, and *w/o Token-Cost* sets $\lambda = 0$.

Long-horizon supervision and adaptive stopping. Both components are important for selecting sufficient rather than merely relevant history. Replacing long-horizon supervision with myopic targets significantly degrades task quality, lengthens profiles, and increases harm, showing that locally

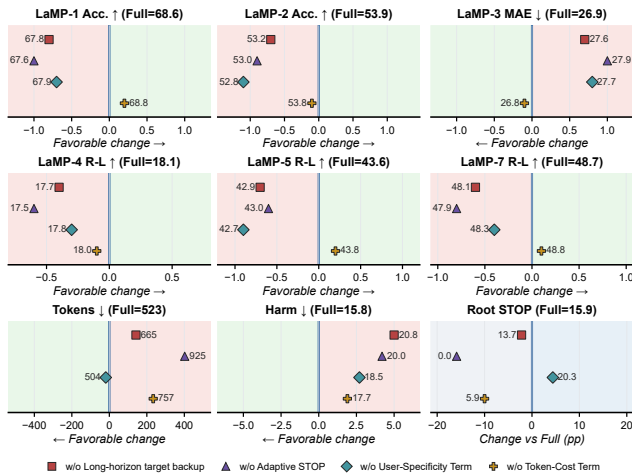


Figure 3: Component ablations. Marker positions show mean changes from full ENOUGH. Rightward is favorable for Acc. and R-L, whereas leftward is favorable for MAE, Tokens, and Harm. Root Stop has no single favorable direction.

attractive records do not reliably form a useful final profile. Disabling adaptive stopping produces the same pattern: fixed-length selection cannot avoid unnecessary evidence. Together, these results validate reasoning about future selections when additional personalization is no longer beneficial.

User specificity and token cost play distinct roles. Removing user specificity significantly reduces task quality though profiles become slightly shorter. Removing token cost yields a slight improvement in aggregate task quality, but significantly increases both token usage while making early stopping much less frequent. The token-cost term is therefore useful: it prevents marginal quality gains from being purchased with disproportionately long and riskier profiles, yielding a better quality–cost–harm trade-off.

3.4 Is More Personalization Always Beneficial?

Longer fixed profiles are not always better. Figure 4(a) shows that fixed-length personalization does not improve monotonically as more history is added. Here, quality aggregates direction-normalized evaluation metrics across all six tasks. Among the evaluated lengths, quality is highest at $k = 5$ (98.7) but falls by 0.8 points at $k = 10$, while harm relative to the empty profile rises from 23.2% to 31.7%. This trade-off motivates instance-adaptive rather than fixed profile lengths. Appendix Table 6 reports the measured quality, token cost, and harm values for the full fixed-length grid.

Overriding predicted Stop becomes increasingly risky. Figure 4(b) forces the controller to append its next one, two, or three records after predicted Stop. From +1 to +3, the beneficial share falls from 18.0% to 9.9%, whereas the harmful share rises from 16.9% to 35.9% alongside increasing token cost. Thus, deeper continuation is less likely to improve the objective and more likely to hurt performance. Appendix Table 7 gives the policy-by-depth breakdown, and Appendix Figure 8 reports calibration of the learned stop confidence to the bounded-tree teacher.

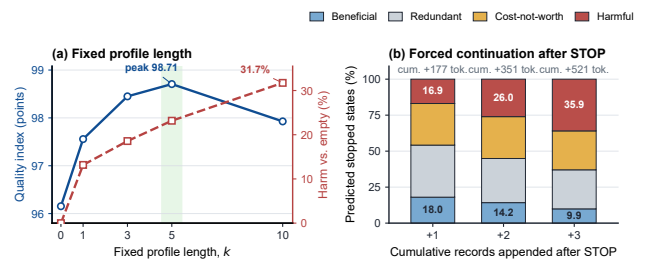


Figure 4: Measured profile-length and stopping diagnostics. (a) Direction-normalized quality and harm rate relative to the empty profile for fixed- k Contriever profiles. (b) Outcome shares after appending one to three controller-selected records beyond predicted Stop prefixes.

Intervention	Quality	Ref.-LL gain	Δ target	Match SMD
Target-user profile	100.0	+0.041	+0.001	0.002
Matched other user	99.1	+0.024	-0.018	0.041
Action shuffle	98.7	+0.018	-0.025	0.039
Owner permutation	98.6	+0.011	-0.029	0.044
Random other user	98.5	+0.012	-0.031	0.180
Whole-profile swap	98.0	+0.005	-0.037	0.191

Table 2: Frozen-generator specificity interventions. Quality is normalized so the target-user profile is 100; likelihood values are per target token.

3.5 User Specificity

Content-matched other-user replacement shifts the normalized quality index from 100.0 to 99.1 and reference-likelihood gain from 0.041 to 0.024 per token (Table 2). This matched audit covers 87.5% of records (mean absolute standardized difference: 0.040), excluding unmatched cases. Together with the $\beta = 0$, context-only, action-only, and no-harm-gate rows, these interventions assess whether the observed gains arise from target-user-specific information rather than generic contextual utility.

Human evaluation. In the human evaluation on LaMP-4 and LaMP-7, the observed win-rate ranges are 53.7–56.0% for task faithfulness, 60.3–61.3% for user-style consistency, and 58.7% overall, with Fleiss’ κ spanning 0.41–0.47 (Appendix E.5). The study randomized presentation order, concealed method identity, and adjudicated only protocol violations; ties were retained as valid outcomes.

3.6 Hyperparameter Sensitivity

We examine how the two objective weights shape distinct aspects of profile construction. We vary β and λ one at a time; at each setting, we hold the remaining configuration fixed, recompute tree backups, and train the corresponding controller. Figure 5 pairs the direction-normalized quality index across all six tasks (higher is better) with specificity gain for β and profile length for λ .

The sweeps motivate the selected operating point $(\beta, \lambda) = (0.4, 0.10)$. In the β sweep, $\beta = 0.4$ gives the highest quality index (99.9) while reaching a specificity gain of 0.90, close

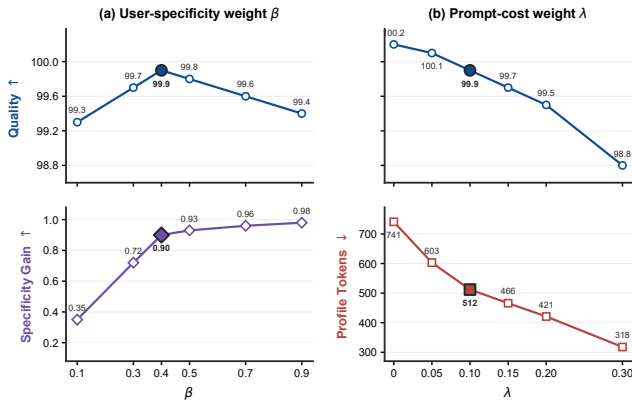


Figure 5: Hyperparameter sensitivity of ENOUGH to β (specificity weight, left) and λ (token-cost weight, right).

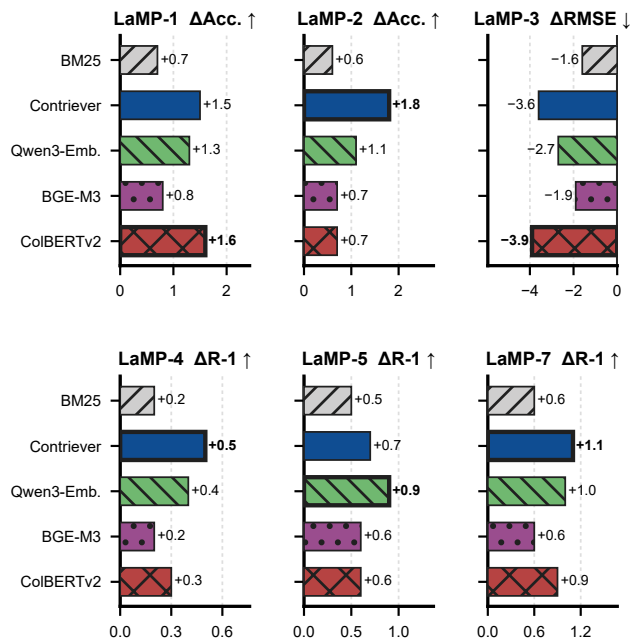


Figure 6: Performance gaps between ENOUGH and the strongest compatible baseline across different retrievers.

to the observed maximum of 0.98. In the λ sweep, $\lambda = 0.10$ reduces mean profile length from 741 to 512 tokens (30.9%) relative to $\lambda = 0$, while the quality index changes from 100.2 to 99.9. Additional development-set sweeps over M , $K_{\max}$, K^- , (J, W) , and the training-loss weights η and ζ are reported in Appendix E.8.

3.7 Retriever Independence

We test whether the gains of ENOUGH are specific to Contriever by repeating the evaluation with BM25, Contriever, Qwen3-Embedding-0.6B (Zhang et al. 2025a), BGE-M3 (Chen et al. 2024), and ColBERTv2 (Santhanam et al. 2022). For each retriever, ENOUGH and all compatible baselines use the same retriever-specific candidate pool and gen-

Method	Quality	Tokens	Calls	p50 ms	p95 ms
ENOUGH	100.00	510	1	370	440
PURPLE	96.49	905	1	398	474
RankGPT	96.32	869	6	1120	1379
REPLUG	95.38	1388	5	1323	1595
IC-RALM	94.82	1210	7	1463	1793
ICR	94.30	819	4	865	1049
RSPG-Pre	94.29	765	3	796	951
Contriever	92.17	931	1	365	442
BM25	90.96	905	1	360	421

Table 3: Measured online inference cost under identical generation settings. Tokens is the mean selected-profile length; Calls is the number of generator-equivalent invocations; p50/p95 are end-to-end latency in milliseconds.

eration settings. Figure 6 reports the resulting test-set gap, $\Delta = \text{ENOUGH} - \text{strongest baseline}$. ENOUGH outperforms the strongest baseline in all 30 retriever–task combinations. Across retrievers, the Accuracy and ROUGE-1 gains range from +0.2 to +1.8 points. The largest margin occurs under different retrievers across tasks, suggesting that the benefit of ENOUGH comes from its selection and stopping strategy rather than from a particular retrieval model.

3.8 Efficiency

Table 3 shows that ENOUGH achieves the highest six-task normalized Quality (100.00) with the shortest selected profile (510 tokens) and one generator-equivalent call. Compared with the strongest baseline, PURPLE, it improves Quality by 3.51 points while using 43.6% fewer profile tokens and reducing p50/p95 latency from 398/474 to 370/440 ms. BM25 and Contriever have comparable one-call latency but trail ENOUGH by 9.04 and 7.83 Quality points, respectively. Multi-call baselines require 3–7 calls and exhibit substantially higher p50/p95 latency (796–1463/951–1793 ms), indicating that the Quality gain of ENOUGH does not rely on additional online calls. Appendix E.10 separate the offline training-time and task-family latency views.

4 Conclusion

This paper introduced ENOUGH, a framework for minimal sufficient personalization that jointly decides whether to personalize, which behavioral records to include, and when to stop. By distilling offline counterfactual evaluations of downstream gain, user specificity, and token cost into a long-horizon controller, ENOUGH constructs input-adaptive profiles while requiring only one frozen-generator call at inference. Across six LaMP tasks and two generator backbones, ENOUGH achieves the best result in all 24 task–metric settings; it also improves normalized quality by 3.51 points over the strongest baseline while using 43.6% fewer profile tokens. These results show that effective personalization depends on identifying when selected user evidence is sufficient, not on exposing models to more history. Future work will extend this formulation to evolving histories and interactive settings.

References

- Chen, J.; Xiao, S.; Zhang, P.; Luo, K.; Lian, D.; and Liu, Z. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2318–2335. Bangkok, Thailand: Association for Computational Linguistics.
- Chen, S.; Jimenez Gutierrez, B.; and Su, Y. 2025. Attention in Large Language Models Yields Efficient Zero-Shot Re-Rankers. In *The Thirteenth International Conference on Learning Representations*.
- Du, L.; Yuan, Y.; Zhao, Z.; Lyu, F.; Penaloza, E.; Chen, X.; Sun, Z.; Kang, J.; Charlin, L.; Liu, X.; and Wu, H. 2026. Optimizing User Profiles via Contextual Bandits for Retrieval-Augmented LLM Personalization. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 31803–31817. San Diego, California, United States: Association for Computational Linguistics.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
- Izacard, G.; Caron, M.; Hosseini, L.; Riedel, S.; Bojanowski, P.; Joulin, A.; and Grave, E. 2022. Unsupervised Dense Information Retrieval with Contrastive Learning. *Transactions on Machine Learning Research*.
- Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents.
- Ram, O.; Levine, Y.; Dalmedigos, I.; Muhlgay, D.; Shashua, A.; Leyton-Brown, K.; and Shoham, Y. 2023. In-Context Retrieval-Augmented Language Models. *Transactions of the Association for Computational Linguistics*, 11: 1316–1331.
- Robertson, S. E.; and Zaragoza, H. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4): 333–389.
- Salemi, A.; Killingback, J.; and Zamani, H. 2025. ExPerT: Effective and Explainable Evaluation of Personalized Long-Form Text Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, 17516–17532. Vienna, Austria: Association for Computational Linguistics.
- Salemi, A.; Mysore, S.; Bendersky, M.; and Zamani, H. 2024a. LaMP: When Large Language Models Meet Personalization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7370–7392. Bangkok, Thailand: Association for Computational Linguistics.
- Salemi, A.; et al. 2024b. Optimization Methods for Personalizing Large Language Models through Retrieval Augmentation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 752–762. Washington, DC, USA: Association for Computing Machinery.
- Santhanam, K.; Khattab, O.; Saad-Falcon, J.; Potts, C.; and Zaharia, M. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 3715–3734. Seattle, United States: Association for Computational Linguistics.
- Shi, T.; Xu, J.; Zhang, X.; Zang, X.; Zheng, K.; Song, Y.; and Li, H. 2025. Retrieval Augmented Generation with Collaborative Filtering for Personalized Text Generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1294–1304. Padua, Italy: Association for Computing Machinery.
- Shi, W.; Min, S.; Yasunaga, M.; Seo, M.; James, R.; Lewis, M.; Zettlemoyer, L.; and Yih, W.-t. 2024. REPLUG: Retrieval-Augmented Black-Box Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 8371–8384. Mexico City, Mexico: Association for Computational Linguistics.
- Sun, W.; Yan, L.; Ma, X.; Wang, S.; Ren, P.; Chen, Z.; Yin, D.; and Ren, Z. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 14918–14937. Singapore: Association for Computational Linguistics.
- Tan, Z.; Zeng, Q.; Tian, Y.; Liu, Z.; Yin, B.; and Jiang, M. 2024. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-tuning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 6476–6491. Miami, Florida, USA: Association for Computational Linguistics.
- Xu, Y.; Zhang, J.; Salemi, A.; Hu, X.; Wang, W.; Feng, F.; Zamani, H.; He, X.; and Chua, T.-S. 2025. Personalized Generation In Large Model Era: A Survey. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 24607–24649. Vienna, Austria: Association for Computational Linguistics.
- Zhang, J. 2024. Guided Profile Generation Improves Personalization with Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 4005–4016. Miami, Florida, USA: Association for Computational Linguistics.
- Zhang, Y.; Li, M.; Long, D.; Zhang, X.; Lin, H.; Yang, B.; Xie, P.; Yang, A.; Liu, D.; Lin, J.; Huang, F.; and Zhou, J. 2025a. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176*.
- Zhang, Z.; Rossi, R. A.; Kveton, B.; Shao, Y.; Yang, D.; Zamani, H.; Derroncourt, F.; Barrow, J.; Yu, T.; Kim, S.; Zhang, R.; Gu, J.; Derr, T.; Chen, H.; Wu, J.; Chen, X.; Wang, Z.; Mitra, S.; Lipka, N.; Ahmed, N. K.; and Wang, Y. 2025b. Personalization of Large Language Models: A Survey. *Transactions on Machine Learning Research*.

A Related Work

A.1 Personalized Large Language Models

Personalized large language models adapt generation to user histories, preferences, or interaction patterns through in-context records, explicit profiles, or user-specific parameters (Zhang et al. 2025b; Xu et al. 2025). Guided Profile Generation distills sparse personal context into concise natural-language descriptions (Zhang 2024), whereas OPPU assigns each user a parameter-efficient module and combines this parametric memory with retrieved profile information (Tan et al. 2024). These approaches provide history abstraction or persistent user modeling. In contrast, our setting freezes both retriever and generator, represents history as behavioral context-action records, and shares a task-conditioned controller across users and tasks. ENOUGH constructs a transparent ordered profile for each input, maintains no user-specific parameters, and may select no record when personalization is not worth its prompt-token cost.

Reliable personalization must also separate user-specific benefit from generic task improvement. ExPerT evaluates content and writing-style alignment (Salemi, Killingback, and Zamani 2025), while CFRAG uses other users’ histories as generation evidence (Shi et al. 2025). Because a generically useful in-context example can improve reference likelihood even when its owner is irrelevant, ENOUGH combines gain over the empty profile with an offline matched-replacement comparison. For each selected slot, it contrasts the target-user record with other-user records matched on task, pre-action content, query relevance, length, and time when available, while holding the remaining profile fixed. Specificity is credited only if the complete target-user profile also improves over no personalization. Matched records supervise the controller but are discarded at inference; the resulting score estimates a replacement effect for the frozen generator, not a causal effect of user preference.

A.2 Retrieval-Augmented Personalized Large Language Models

Retrieval-augmented personalization conditions a language model on selected user-history records without updating the generator for each user. LaMP established this setting across classification and generation tasks (Salemi et al. 2024a). Later work optimizes selection for downstream utility: RSPG chooses among no retrieval and multiple personalized retrievers using generation feedback (Salemi et al. 2024b); CFRAG expands the evidence pool with similar users and learns retrieval and reranking (Shi et al. 2025); and PURPLE trains an order-sensitive Plackett–Luce profile policy from reference-response likelihood (Du et al. 2026). General-purpose rerankers RankGPT (Sun et al. 2023) and ICR (Chen et al. 2025) provide strong pool-reordering baselines but do not model profile termination. CFRAG returns a top- k set, PURPLE an ordered K -permutation, and our LaMP adaptations of RankGPT and ICR a preset number of records. RSPG may choose no retrieval but does not make record-wise continuation decisions after selecting a retriever. ENOUGH instead treats record inclusion and Stop as actions in one sequential problem after frozen candidate retrieval. It can

return an empty profile or any feasible length up to $K_{\max}$, evaluating terminal profiles by downstream gain, incremental prompt-token cost, and user-specific benefit.

Other retrieval-augmented systems alter how evidence enters generation: IC-RALM switches context during decoding (Ram et al. 2023), whereas REPLUG marginalizes predictions from multiple record-conditioned streams (Shi et al. 2024). These methods primarily address knowledge-centric evidence and use inference paths different from personalized profile construction. ENOUGH makes all inclusion and stopping decisions before generation with a lightweight controller trained from bounded-tree outcomes, then invokes the frozen generator exactly once; a root Stop means no profile is injected, although candidate retrieval has already occurred.

B Notation

Symbol	Meaning
C_i	top- M candidate records from the frozen retriever
S_t	ordered profile after t inclusion actions
$g_i(\mathbf{S})$	downstream likelihood gain over the empty profile
$p_i(\mathbf{S})$	matched-replacement user specificity
$c_i(\mathbf{S})$	normalized incremental prompt-token cost
$U_i(\mathbf{S})$	cost-free personalized utility
$J_i^\lambda(\mathbf{S})$	cost-regularized terminal objective
$Q_\theta^{g,p,c}$	predicted outcome components of one action
STOP	terminate with the current profile

Table 4: Core notation.

C Algorithms

Algorithm 1 Offline bounded-tree supervision

Require: Training data $\mathcal{D}$; frozen retriever R ; frozen generator G ; candidate limit M ; horizon $K_{\max}$; search budgets J, W

Ensure: Labeled state-action set $\mathcal{Z}$

- 1: $\mathcal{Z} \leftarrow \emptyset$
- 2: **for** each training instance i **do**
- 3: $C_i \leftarrow R(x_i, H_{u_i}; M)$ and remove target duplicates
- 4: Precompute fixed matched replacements for every $d \in C_i$
- 5: Sample partial-profile roots from the current state-proposal mixture
- 6: **for** each sampled root state s **do**
- 7: Build bounded prefix tree $\mathcal{T}_i(s)$ with budgets J, W
- 8: Score every unique terminal prefix and its sampled slot replacements
- 9: Back up J_i^λ from leaves to expanded actions
- 10: **for** each labeled (s', a) in $\mathcal{T}_i(s)$ **do**
- 11: Propagate the three components of the same best leaf
- 12: $\mathcal{Z} \leftarrow \mathcal{Z} \cup \{(s', a, \mathbf{R}_i(s', a))\}$
- 13: **end for**
- 14: **end for**
- 15: **end for**
- 16: **return** $\mathcal{Z}$

Offline complexity. With N_s sampled roots, horizon $K_{\max}$, search width W , action budget J , and K^- matched replacements, offline label construction requires at most

$$O(N_s K_{\max} W J (K^- + 2)) \quad (33)$$

teacher-forced scoring operations before prefix caching. The $K^- + 2$ factor accounts for the target profile, the leave-one-out profile, and the matched replacements for the sampled slot. These claims assume cached candidate representations and do not include the frozen candidate-retrieval cost.

Algorithm 2 Minimal sufficient profile construction

Require: Input (τ, x, H_u) ; frozen retriever R ; controller Q_θ ; frozen generator G

Ensure: Ordered profile $\hat{\mathbf{S}}$ and output $\hat{y}$

- 1: $C \leftarrow R(x, H_u; M)$; $\mathbf{S} \leftarrow \emptyset$
- 2: **while** $|\mathbf{S}| < \min(K_{\max}, |C|)$ **do**
- 3: Mask repeated records and actions exceeding $L_{\max}$
- 4: Score all feasible record actions and STOP in one controller batch
- 5: $a \leftarrow \arg \max_{a' \in \mathcal{A}(s)} Q_\theta^{\text{net}}(s, a')$
- 6: **if** $a = \text{STOP}$ **then**
- 7: **break**
- 8: **end if**
- 9: $\mathbf{S} \leftarrow \mathbf{S} \oplus a$
- 10: **end while**
- 11: $\hat{y} \leftarrow G(P_\tau(x, \mathbf{S}))$
- 12: **return** $(\mathbf{S}, \hat{y})$

Test-time procedure. Algorithm 2 gives the inference procedure. Candidate record embeddings are cached and all remaining actions are scored in a batch at each step. Neither record selection nor the STOP decision uses a reference output, matched replacement, generator likelihood, counterfactual rollout, or generator call. The controller performs at most $K_{\max}$ lightweight decision steps. Once profile construction terminates, the autoregressive generator is called exactly once on the final prompt.

Online complexity. Online profile construction uses at most $O(K_{\max} M)$ batched lightweight action scores and exactly one final generator call. This bound assumes cached candidate representations and excludes the frozen candidate-retrieval cost.

D Proofs

D.1 Proposition 1: Exact-value stopping

Proof. Fix an instance i and a reachable state s whose current ordered profile prefix is $\mathbf{S}$. Let $\mathcal{L}_i(\mathbf{S})$ be the set of terminal profile prefixes reachable from $\mathbf{S}$, including $\mathbf{S}$ itself via the STOP action. Because C_i is finite, records cannot repeat, and profile length is at most K_i , every candidate subtree is finite. Let $h(\mathbf{S})$ denote the maximum number of record-action edges on a path starting from $\mathbf{S}$.

We first establish, by induction on $h(\mathbf{S})$, that

$$V_i^*(\mathbf{S}) = \max_{\mathbf{L} \in \mathcal{L}_i(\mathbf{S})} J_i^\lambda(\mathbf{L}). \quad (34)$$

If $h(\mathbf{S}) = 0$, no record action is feasible, so $\mathcal{L}_i(\mathbf{S}) = \{\mathbf{S}\}$ and the boundary condition gives $V_i^*(\mathbf{S}) = J_i^\lambda(\mathbf{S})$. Now suppose $h(\mathbf{S}) > 0$ and that Eq. (34) holds for every child prefix. The terminal descendants decompose as

$$\mathcal{L}_i(\mathbf{S}) = \{\mathbf{S}\} \cup \bigcup_{d \in \mathcal{A}(s) \setminus \{\text{STOP}\}} \mathcal{L}_i(\mathbf{S} \oplus d). \quad (35)$$

Applying the induction hypothesis to each child and then

Task	#Train	#Dev	#Test	Input length	Output length	Profile size	#Labels
LaMP-1: Personalized Citation Identification	6,542	1,500	1,500	51.44 ± 5.71	–	84.16 ± 47.54	2
LaMP-2: Personalized Movie Tagging	5,073	1,410	1,557	92.40 ± 21.95	–	86.76 ± 189.53	15
LaMP-3: Personalized Product Rating	20,000	2,500	2,500	128.18 ± 146.25	–	185.41 ± 129.30	5
LaMP-4: Personalized News Headline Generation	12,500	1,500	1,800	29.98 ± 12.09	10.08 ± 3.11	204.60 ± 250.75	–
LaMP-5: Personalized Scholarly Title Generation	14,682	1,500	1,500	162.34 ± 65.64	9.72 ± 3.21	87.89 ± 53.64	–
LaMP-7: Personalized Tweet Paraphrasing	13,437	1,498	1,500	29.72 ± 7.01	16.97 ± 5.68	15.71 ± 14.86	–

Table 5: Statistics of the LaMP time-based (T) split used in our main experiments. Counts are instances; input and output lengths are whitespace-delimited words; profile size is the number of historical records. Length and profile statistics are mean $\pm$ standard deviation over all train, development, and test instances. Classification and rating outputs are labels, so their output lengths are omitted.

Eq. (20) yields

$$\max_{\mathbf{L} \in \mathcal{L}_i(\mathbf{S})} J_i^\lambda(\mathbf{L}) = \max \left\{ J_i^\lambda(\mathbf{S}), \max_{d \in \mathcal{A}(s) \setminus \{\text{STOP}\}} V_i^*(\mathbf{S} \oplus d) \right\} \quad (36)$$

$$= V_i^*(\mathbf{S}), \quad (37)$$

which proves Eq. (34).

Consider now a policy that selects an action maximizing $Q_i^*(s, a)$. By the definitions of the action values and the Bellman recursion,

$$\max_{a \in \mathcal{A}(s)} Q_i^*(s, a) = V_i^*(\mathbf{S}). \quad (38)$$

If the policy selects STOP, then $J_i^\lambda(\mathbf{S}) = Q_i^*(s, \text{STOP}) = V_i^*(\mathbf{S})$, so Eq. (34) shows that the current prefix is optimal among all terminal descendants. If it selects a record d , then $V_i^*(\mathbf{S} \oplus d) = Q_i^*(s, d) = V_i^*(\mathbf{S})$. The child subtree has smaller height, so repeating the same argument eventually reaches a terminal prefix $\mathbf{L}$ with $J_i^\lambda(\mathbf{L}) = V_i^*(\mathbf{S})$. Termination is guaranteed because every record action descends one level in a finite tree.

When STOP is tied with a record action, it is itself a maximizing action; the STOP case above therefore shows that resolving the tie in its favor preserves optimality. Applying the result at the root proves the proposition. $\square$

This guarantee is conditional on access to the exact values Q_i^* . It does not imply that the learned controller Q_θ recovers the optimum of the complete combinatorial candidate tree.

E Supplementary Experimental Details

E.1 Datasets, Time-Based Split Protocol, and Evaluation

We use the six publicly available tasks in the official LaMP snapshot: LaMP-1–5 and LaMP-7. LaMP-6 is excluded because its publicly distributed files contain only identifiers from the licensed Avocado email collection rather than the underlying email text. For the main experiments, we use the time-based (T) protocol, in which interactions are ordered chronologically: earlier records form the profile and later interactions form the prediction targets, thereby evaluating

personalization for future interactions of existing users. Table 5 reports time-based statistics computed directly from the files used in the main experiments.

E.2 Implementation Details

Models and search. Our frozen retriever is Contriever, which returns at most $M = 20$ legal records in the main setting. We use Qwen3.5-9B as the main LLM and Llama-3.1-8B-Instruct for generator transfer; both LLMs are frozen and use deterministic decoding with their respective official chat templates. Qwen3.5-9B is served through its text-only path with `enable_thinking=false`. Teacher-forced likelihood is computed only over reference-response tokens, excluding prompt tokens and all chat-template, control, and thinking tokens. The controller has $K_{\max} = 10$, uses $K^- = 3$ matched alternatives per candidate, and is trained from bounded-tree targets with the policy-induced offline aggregation described below. The search uses $J = 8$ candidate expansions and beam width $W = 4$ at the default operating point. We set the ranking- and stopping-loss weights in Eq. (31) to $\eta = 0.2$ and $\zeta = 0.4$, respectively. Frozen-model likelihoods and tokenized record encodings are cached across selectors and λ values. For each generator, we independently recompute tokenizer costs, likelihoods, controller labels, and caches rather than reusing artifacts produced by another model.

Policy-induced offline aggregation. Algorithm 1 is the label-construction routine used for both rounds of offline supervision. We first apply it to the initial state-proposal mixture and optimize the controller under Eq. (31). After warm-start convergence, one policy-induced offline data-aggregation round adds states visited by the current controller, reuses the same frozen generator to label them, and retrain the controller on the union of both state pools. No controller update is performed during test-time profile construction.

Baseline configurations. BM25 and Contriever independently retrieve the top $K = 5$ records from the same leakage-filtered user history using the official task-specific query fields. RankGPT, ICR, and PURPLE receive the same top $M = 20$ pool from frozen Contriever and return

$K = 5$ records; following the published LaMP adaptation, RankGPT and ICR use frozen Llama-3.1-8B-Instruct as a separate reranker. IC-RALM and REPLUG use Contriever evidence under the same five-record budget but retain their native context-switching and marginalization mechanisms; for REPLUG, we use the LM-supervised retrieval variant from its LaMP adaptation. RSPG-Pre operates over its native retriever pool and is retrained with our task generator and $K = 5$. ENOUGH and the single-prompt baselines invoke the task generator once after profile construction, whereas IC-RALM and REPLUG retain their native multi-pass inference. All methods share the record serialization, task prompt, frozen task generator, decoding configuration, and leakage controls. Empty/full profiles, fixed- k and fixed-token controls, Recency, Random, MMR, ROPG-KD, the myopic controller, and forced-length ENOUGH variants are retained as diagnostics or ablations rather than as main baselines.

Serialization, leakage controls, and matching. Each profile record is serialized with a task tag and explicit field boundaries:

```
[TASK]  $\tau$  [HIST_CONTEXT]  $x_d$ 
[USER_ACTION]  $a_d$  [META]  $m_d$ .
```

The current input is serialized separately and never copied into the profile. We preserve complete record boundaries, mask an action if its complete serialization would overflow the context window, and never silently truncate records. Within each generator condition, all selectors use the same generator-specific tokenizer and prompt construction. The empty-profile prompt contains the same instructions and delimiters but no historical record, so its comparison changes only the augmentation content. Appendix F shows the complete logical system and user messages for all six tasks.

For every target, the legal user history contains only interactions whose timestamps strictly precede the target timestamp. We apply this temporal truncation before constructing the retrieval pool or drawing matched replacements. Before retrieval, we remove the target interaction, matching source IDs, normalized exact duplicates, and embedding-near duplicates above a development-set threshold; consequently, neither retrieval nor matching can access an interaction or metadata recorded at or after the target. Matched other-user records come from the same task and legal partition and are matched using pre-action semantic content, length, retrieval-score bin, time bin, and available nonbehavioral metadata. Historical actions, target references, and all post-target fields are unavailable to the matcher. The matched-specificity test therefore estimates a frozen-LLM replacement effect and is not a causal claim about users.

Cost and statistical protocol. Profile cost is the actual incremental number of tokenizer tokens, not document count. We additionally record selected records, root STOP rate, harm rate relative to the empty-profile output, generator calls, latency, and throughput. A root STOP means *no profile augmentation*; candidate retrieval has still occurred. Every reranker and generator-equivalent forward pass is included in the cost audit.

We choose the operating point, β , λ , loss weights, temperatures T_r and T_s , calipers, M , $K_{\max}$, K^- , J , and W using

development data only; no test result is used for tuning, and all selected values are frozen before test evaluation. Each automatic result uses three complete pipeline seeds. We pair predictions at the instance level, use a paired 95% bootstrap clustered by user rather than by isolated record, and apply Holm-corrected comparisons against all main baselines. Reported seed variation reflects variation across complete experimental runs.

E.3 Generator-Scale Sensitivity

We evaluate task-generator capacity on the time-based (T) split by comparing dense Qwen3.5-4B, Qwen3.5-9B, and Qwen3.5-27B (Qwen Team 2026) generators while holding the retriever pool, search budget, task prompt, deterministic decoding, and leakage controls fixed. Tokenizer costs, frozen-generator likelihoods, bounded-tree labels, caches, and the controller are recomputed independently for each scale; no controller or generator-dependent artifact is transferred from the 9B condition.

Figure 7 reports both official metrics for every task and all nine main methods. The 9B column reproduces the corresponding main-table results, while the 4B and 27B columns report independently measured scale conditions. Every cell aggregates three complete pipeline seeds. As in Table 1, every displayed metric is multiplied by 100.

The measured scale response differs across methods, tasks, and metrics rather than following a uniform offset. From 4B to 27B, baseline changes range from 1.2–2.5 points for classification accuracy, 1.2–2.6 for F1, 1.3–2.7 for MAE reduction, 2.5–5.0 for RMSE reduction, and 0.4–2.0 for ROUGE. The consistent gains show that greater generator capacity generally improves downstream quality, while the heterogeneous magnitudes indicate that scale does not affect every task or selector equally.

(a) LaMP-1 · Acc. ↑				LaMP-1 · F1 ↑				(b) LaMP-2 · Acc. ↑				LaMP-2 · F1 ↑			
BM25	62.3	63.1	63.5	60.7	61.5	62.0		49.4	50.2	50.6		45.0	45.8	46.2	
Contriever	63.6	64.5	65.1	62.2	63.0	63.8		48.7	49.8	50.2		43.9	44.9	45.4	
IC-RALM	65.7	67.1	68.0	64.0	65.6	66.5		48.8	50.1	51.0		44.5	46.0	46.8	
REPLUG	64.1	65.7	66.6	62.7	64.1	65.3		47.1	48.7	49.4		43.1	44.7	45.5	
RankGPT	65.6	67.0	68.0	63.6	65.3	66.2		48.9	50.2	51.2		44.4	46.0	46.9	
ICR	63.8	64.8	65.4	62.5	63.4	64.2		49.9	51.2	51.7		45.4	46.6	47.1	
RSPG-Pre	65.2	66.2	66.9	63.6	64.7	65.4		49.4	50.3	51.0		45.1	46.1	46.7	
PURPLE	65.5	66.7	67.5	64.0	65.1	66.1		50.7	52.1	52.7		45.8	47.0	47.7	
ENOUGH	67.4	68.6	69.4	65.7	67.0	67.9		52.7	53.9	54.6		47.3	48.6	49.3	
	4B	9B	27B	4B	9B	27B		4B	9B	27B		4B	9B	27B	

(c) LaMP-3 · MAE ↓				LaMP-3 · RMSE ↓				(d) LaMP-4 · R-1 ↑				LaMP-4 · R-L ↑			
BM25	32.5	31.7	31.2	70.4	68.8	67.9		18.7	19.0	19.2		16.5	16.7	16.9	
Contriever	32.3	31.4	30.7	69.1	67.6	66.3		18.4	18.8	19.0		16.7	17.0	17.2	
IC-RALM	30.6	29.3	28.3	68.9	66.2	64.6		17.3	17.7	18.2		16.4	16.8	17.1	
REPLUG	30.0	28.5	27.3	64.7	62.4	60.3		18.4	19.1	19.5		17.0	17.5	17.9	
RankGPT	29.6	28.2	27.0	65.0	61.9	60.0		17.8	18.3	18.9		17.1	17.6	18.0	
ICR	31.5	30.5	29.7	68.8	67.0	65.5		18.6	19.1	19.4		17.0	17.3	17.5	
RSPG-Pre	30.3	29.4	28.6	68.3	66.4	65.2		17.9	18.2	18.6		16.9	17.2	17.4	
PURPLE	30.1	29.0	28.1	67.2	65.5	63.7		18.9	19.4	19.7		17.4	17.8	18.1	
ENOUGH	28.1	26.9	25.9	60.6	58.3	56.6		19.5	19.9	20.3		17.7	18.1	18.4	
	4B	9B	27B	4B	9B	27B		4B	9B	27B		4B	9B	27B	

(e) LaMP-5 · R-1 ↑				LaMP-5 · R-L ↑				(f) LaMP-7 · R-1 ↑				LaMP-7 · R-L ↑			
BM25	44.9	45.5	45.9	39.3	39.9	40.3		48.7	49.3	49.7		44.2	44.8	45.2	
Contriever	45.4	46.1	46.6	40.2	40.7	41.3		48.0	48.8	49.2		45.0	45.6	46.1	
IC-RALM	44.8	45.8	46.6	41.2	42.3	43.0		50.7	51.7	52.5		46.0	47.0	47.7	
REPLUG	44.5	45.7	46.5	40.9	41.8	42.7		49.8	51.0	51.7		46.5	47.5	48.3	
RankGPT	46.5	47.5	48.4	41.8	42.8	43.6		48.8	49.8	50.7		45.9	46.9	47.7	
ICR	45.6	46.4	47.0	41.2	41.8	42.4		48.9	49.8	50.2		46.4	47.1	47.6	
RSPG-Pre	45.3	46.1	46.7	40.6	41.3	41.8		48.7	49.4	50.0		45.4	46.1	46.6	
PURPLE	44.4	45.3	46.0	41.8	42.5	43.3		50.8	51.9	52.4		47.0	47.8	48.4	
ENOUGH	47.3	48.2	48.9	42.8	43.6	44.3		52.1	53.0	53.6		47.9	48.7	49.3	
	4B	9B	27B	4B	9B	27B		4B	9B	27B		4B	9B	27B	

Figure 7: Generator-scale sensitivity on the LaMP time-based (T) split, averaged over three complete pipeline seeds. Each task is represented by two adjacent panels, one per official metric, and task-level labels run from (a) through (f); each row therefore contains four panels for two tasks. We compare dense Qwen3.5-4B, Qwen3.5-9B, and Qwen3.5-27B task generators. Cells display raw values; darker shading denotes better performance within each panel after respecting metric direction, so color intensity is not comparable across panels. Higher is better except for MAE and RMSE.

E.4 Profile Length and Stopping Diagnostics

The diagnostics separate length effects from stopping quality. We report three complementary measured diagnostics. Table 6 tests whether a common fixed length benefits all instances; Table 7 tests whether overriding a learned Stop decision remains useful; and Figure 8 tests whether the learned stop score tracks its bounded-tree supervision. Unless stated otherwise, results use the time-based split with Qwen3.5-9B and average three complete pipeline seeds. Quality is the six-task macro-average after orienting each official task metric so that higher is better; token counts are

actual increments over the empty-profile prompt.

Fixed profiles expose a quality-risk trade-off. We first vary the number of records while holding the frozen Contriever retriever, generator, and evaluation protocol fixed. A fixed- k profile contains up to the first k legal retrieved records.

As Table 6 shows, the aggregate quality index rises from 96.15 with no profile to 98.71 at $k = 5$, but falls to 97.93 at $k = 10$. Over the same grid, mean profile cost reaches 1,755 tokens and the harm rate relative to the empty-profile output rises from the defined 0% baseline to 31.7%. Thus,

Length	Quality	Tokens	Harm
k=0	96.15	0	0.0%
k=1	97.56	200	13.2%
k=3	98.45	560	18.6%
k=5	98.71	918	23.2%
k=10	97.93	1755	31.7%

Table 6: Measured fixed-length diagnostic on the time-based split, averaged over three complete pipeline seeds. Profiles contain up to the first k legal Contriever records. Quality is the direction-normalized six-task macro-index; tokens are incremental over the empty-profile prompt; harm is measured relative to the empty-profile output, so $k = 0$ is the defined zero-harm baseline.

quality is non-monotonic within the evaluated grid while both cost and instance-level risk increase with length. This diagnostic motivates instance-adaptive length selection; it does not identify $k = 5$ as a universally optimal length.

Forced continuation isolates the decision value of Stop.

For each stopped prefix with enough remaining legal actions, we suppress Stop and cumulatively append $d \in \{1, 2, 3\}$ records using the controller ordering, the frozen retriever ranking, or a reference-aware oracle-best diagnostic ordering. The $+d$ rows therefore evaluate cumulative continuations from the same prefix rather than the marginal effect of only the d th record; the oracle ordering is diagnostic and is not available at deployment. Relative to stopping at that prefix, outcomes are assigned in priority order: *beneficial* if continuation increases J_i^λ ; among the remaining cases, *cost-not-worth* if direction-normalized task quality improves, *harmful* if it worsens, and *redundant* otherwise. This ordering makes the four categories mutually exclusive and exhaustive before rounding. Because J_i^λ combines task gain, specificity, and token cost, *cost-not-worth* is shorthand for a task-quality gain rejected by the full objective, not evidence that token cost alone caused the rejection.

Table 7 shows that, for controller-ordered continuation, the beneficial share decreases from 18.0% at $+1$ to 9.9% at $+3$, whereas the harmful share increases from 16.9% to 35.9% as the cumulative added cost grows from 177 to 521 tokens. Retriever-ordered continuation is more harmful at every reported depth. The oracle-best diagnostic retains a larger beneficial share—28.2% at $+1$ —which confirms that some learned stops leave useful continuations, but its beneficial share also falls and its harmful share rises with depth. These are descriptive aggregate comparisons: without row-level paired intervals and a common-depth eligibility audit, they do not establish policy dominance or imply that every learned stop is optimal.

The stop confidence tracks the bounded-tree teacher, not an exact oracle. Figure 8 plots $\hat{p}_{\text{stop}}(s) = \sigma(\Delta_\theta^{\text{stop}}(s)/T_s)$, where Eq. (24) defines the controller margin and Eq. (30) defines its temperature-scaled training target. The development-selected T_s is frozen for test evaluation. We evaluate against the hard bounded-teacher decision

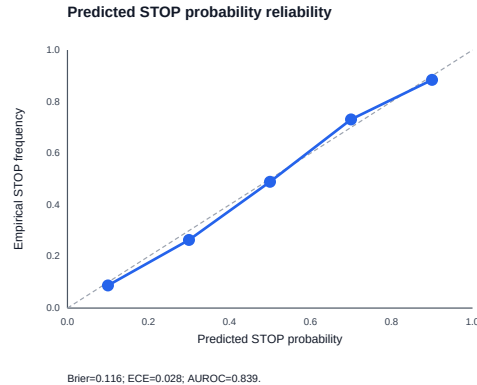


Figure 8: Measured calibration of the temperature-scaled Stop confidence against binary bounded-tree teacher decisions. Each point compares the mean predicted confidence with the teacher-label frequency in one of five bins; the dashed diagonal denotes perfect agreement. This is calibration to the bounded-tree teacher, not to exact global stopping optimality.

$\mathbb{I}[\Delta_{T_s}^{\text{stop}}(s) \geq 0]$, consistent with resolving ties in favor of Stop. The five mean confidence/teacher-frequency pairs are 0.10/0.09, 0.30/0.26, 0.50/0.49, 0.70/0.73, 0.90/0.88. Within this binning, the Brier/ECE scores of 0.116/0.028 indicate close aggregate agreement, while AUROC/AUPRC scores of 0.839/0.794 indicate useful discrimination. These metrics do not estimate the probability that Stop is globally optimal, and bin counts or uncertainty intervals are still needed to assess sparsely populated regions.

Residual stopping errors occur in both directions. The error audit records a stop as premature when the best available exact or bounded-teacher continuation value strictly exceeds the Stop value, as late when the teacher preferred Stop at an earlier prefix, and as forced when the controller reaches K_{max} without selecting Stop. These are separate trajectory events and are not assumed to partition the diagnostic subset. Their respective rates are 8.8%, 10.7%, and 3.0%, and the associated mean utility loss relative to the audit’s continuation benchmark is 0.076. The late-stop rate is numerically 1.9 percentage points higher than the premature-stop rate, but without paired uncertainty we treat this difference as descriptive rather than evidence of a systematic error asymmetry. Because this audit can use bounded-tree continuation values, its stopping-regret statistic should not be conflated with the reduced-space exact-oracle terminal-profile regret in Appendix E.7. Overall, the diagnostics support Stop as an informative, imperfect decision signal relative to its teacher; they do not establish globally minimal profiles.

Continuation	Cum. depth	Cum. extra tok.	Beneficial	Redundant	Cost-not-worth	Harmful
controller-next	+1	177	18.0%	36.2%	28.9%	16.9%
controller-next	+2	351	14.2%	30.7%	29.1%	26.0%
controller-next	+3	521	9.9%	27.1%	27.0%	35.9%
retriever-next	+1	181	14.0%	33.6%	29.0%	23.4%
retriever-next	+2	359	9.7%	29.5%	28.0%	32.9%
retriever-next	+3	536	7.2%	23.8%	25.0%	43.9%
oracle-best-continuation	+1	170	28.2%	33.6%	26.2%	11.9%
oracle-best-continuation	+2	331	21.8%	30.7%	29.3%	18.2%
oracle-best-continuation	+3	490	17.8%	29.1%	28.0%	25.1%

Table 7: Measured forced-continuation diagnostic from prefixes where Stop was predicted, averaged over three complete pipeline seeds. For each policy, $+d$ cumulatively appends its first d records from the same prefix; extra tokens are cumulative relative to that prefix. Categories partition the evaluated stopped prefixes before rounding.

Audit	Coverage	Reject	Exact	Mean SMD	p90 SMD
$K^- = 3$; caliper enabled	87.5%	8.1%	31.1%	0.040	0.083

Table 8: Measured balance audit for content-matched replacements. Coverage is the share of selected records with three valid controls; Reject is the share excluded by the caliper.

E.5 Specificity, Matching, and Human Evaluation

We assess whether the selected profiles capture target-user behavior rather than merely providing useful task demonstrations from three complementary perspectives: a balance audit of the matched controls, frozen-generator interventions on profile ownership, action, and content, and blind human judgments of the resulting outputs.

Matching quality bounds the specificity analysis. Table 8 reports both the feasibility of constructing matched controls and their balance on variables observed before the user action. Coverage is the fraction of selected records for which all $K^- = 3$ other-user controls satisfy the caliper; candidates that fail this requirement are rejected rather than silently replaced. Exact-match rate measures agreement within discrete matching strata, whereas standardized mean differences summarize balance over continuous covariates. Because the subsequent replacement comparisons exclude unmatched records, they characterize the matched subset rather than all selected profiles.

Interventions distinguish personal from generic utility. Table 2 compares the target-user profile with five controls. Content-matched replacement approximately preserves observable content and query relevance while replacing a target-user record with a record from another user; random-user replacement and whole-profile swaps provide progressively less constrained controls. Action shuffling breaks the pairing between a historical context and the action taken in that context. Finally, owner permutation keeps record contents fixed while reassigning ownership labels, serving as

Task	Criterion	Win	Tie	Loss	κ
LaMP-4	task faithfulness	56.0%	24.0%	20.0%	0.41
LaMP-4	user style consistency	61.3%	21.0%	17.7%	0.47
LaMP-4	overall preference	58.7%	23.0%	18.3%	0.44
LaMP-7	task faithfulness	53.7%	25.3%	21.0%	0.41
LaMP-7	user style consistency	60.3%	22.0%	17.7%	0.44
LaMP-7	overall preference	58.7%	24.0%	17.3%	0.42

Table 9: Measured three-annotator blind comparison of ENOUGH against PURPLE on 200 samples per task. Win, tie, and loss are defined from the perspective of ENOUGH; κ denotes Fleiss’ agreement coefficient.

a negative-control audit of whether the matching pipeline itself can manufacture an apparent specificity effect. These comparisons estimate frozen-generator replacement effects conditional on the achieved covariate balance; they do not identify a causal effect of user identity.

Blind human judgments test whether the differences are perceptible. For each of LaMP-4 and LaMP-7, Table 9 compares ENOUGH with PURPLE on 200 examples, with three annotators independently judging each example. Task faithfulness, consistency with the user’s style, and overall preference are elicited as separate questions so that task correctness is not conflated with stylistic personalization. Presentation order is randomized and method identity is concealed. Annotators see the current input and the legally available user history, but not likelihood scores; automatic references are also hidden for the overall-preference judgment. Ties are retained as valid outcomes, and adjudication is restricted to protocol violations. The table reports win/tie/loss rates and Fleiss’ κ from the completed annotation study.

E.6 Additional Ablations

The main ablation figure isolates long-horizon targets, adaptive stopping, user specificity, and token cost. We additionally test the value-head decomposition, ranking and STOP losses, and order-sensitive profile encoding.

Variant	Quality	Tokens	Harm	Regret
Full ENOUGH	99.9	512	15.0%	0.070
Single net-value head	99.4	515	17.1%	0.096
No ranking loss	99.4	516	16.7%	0.090
No STOP loss	99.0	581	18.7%	0.121
Permutation-invariant profile	99.2	513	17.5%	0.112

Table 10: Measured controller-objective and representation ablations, averaged over three complete pipeline seeds. Higher Quality and lower token cost, harm, and regret are better.

Selector	Regret	ϵ -suff.	Excess tok.	Stop agree
Exact oracle	0.000	100.0%	0	100.0%
Bounded-tree teacher	0.028	91.6%	41	92.3%
Learned ENOUGH	0.069	81.8%	89	83.7%
Myopic	0.129	68.5%	169	71.9%
Fixed $k = 4$	0.152	64.1%	248	65.2%

Table 11: Measured exact-oracle diagnostic pooled over 50 examples per task with $M' = 6$ and $K' = 4$. Lower is better for regret and excess tokens; higher is better for ϵ -sufficiency and stop agreement.

Task	Teacher	ENOUGH	Myopic	Fixed-4
LaMP-1	0.025	0.064	0.122	0.147
LaMP-2	0.030	0.073	0.130	0.154
LaMP-3	0.042	0.082	0.141	0.162
LaMP-4	0.024	0.066	0.126	0.148
LaMP-5	0.037	0.080	0.137	0.161
LaMP-7	0.023	0.065	0.121	0.144

Table 12: Measured utility regret to the exact oracle by task; lower is better, and the exact oracle has zero regret by definition.

E.7 Exact Minimal-Sufficient Oracle

We measure how closely each selector approaches a minimal sufficient profile using an exact-oracle diagnostic on 50 examples per task. For each instance, we reduce the candidate pool to $M' = 6$, cap the profile length at $K' = 4$, and enumerate all 517 legal ordered profiles: the empty profile and every length-one-to-four permutation drawn without replacement. All methods use the same frozen generator and exact token cost. Because the exact oracle scores profiles using the gold reference, it provides a diagnostic upper bound rather than a deployable selector.

The bounded-tree teacher retains most of the oracle performance, with 0.028 utility regret, 91.6% ϵ -sufficiency, 41 excess tokens, and 92.3% agreement with the oracle stop decision (Table 11). The learned ENOUGH controller obtains 0.069 regret and reaches ϵ -sufficiency on 81.8% of instances while using 89 excess tokens and matching the oracle stop decision 83.7% of the time. It is substantially closer to the oracle than the myopic and fixed- k selectors on all four diagnostics. The gap between the exact oracle and bounded teacher quantifies bounded-search truncation, whereas the

additional gap from the teacher to ENOUGH reflects controller approximation.

The task-level results show the same ordering: ENOUGH regret ranges from 0.064 on LaMP-1 to 0.082 on LaMP-3, below both the myopic and fixed-4 alternatives on every task (Table 12). Pooled quantities are computed directly over diagnostic instances rather than by averaging heterogeneous task-level metrics.

E.8 Additional Hyperparameter Sensitivity

To assess whether ENOUGH depends on a narrow configuration, we conduct additional one-at-a-time sweeps on the development set over the candidate-pool size M , maximum profile depth $K_{\max}$, number of matched controls K^- , offline search budget (J, W) , and auxiliary-loss weights η and ζ . All remaining settings are held fixed at their selected values. For each task, we orient its official development metric so that higher is better and normalize it to a common 100-point reference level; the reported quality index is the macro-average of the six task-level indices rather than an average of raw heterogeneous metrics. Stop regret is the mean utility shortfall of the controller-selected stopping point relative to the best feasible stopping value on the exact-oracle diagnostic subset in Appendix E.7. For the structural and search parameters, we report both metrics; for the loss weights, we report only aggregate quality. Following the protocol above, each plotted point aggregates three complete pipeline seeds. Because the figures do not show seed-level intervals, we interpret the trends descriptively within the evaluated grids and do not use these sweeps to claim statistical significance. All operating points are selected using development data and frozen before test evaluation.

Structural and search parameters. Figure 9 shows that the selected settings lie near favorable regions of their respective sweeps. Among the evaluated pool sizes, $M = 20$ attains both the highest quality value, 99.4, and the lowest stop regret, 0.069; enlarging the pool further produces small degradations in both metrics. Increasing $K_{\max}$ from 2 to 10 substantially improves quality and stop regret, whereas $K_{\max} = 15$ preserves the same quality of 99.4 and lowers regret by only 0.001. Similarly, $K^- = 3$ captures most of the improvement from additional matched controls: moving to four or five controls changes quality by at most 0.1 and stop regret by at most 0.001. Larger search budgets continue to improve both metrics, but the gains diminish beyond $(J, W) = (8, 4)$: increasing the budget to $(16, 8)$ raises quality from 99.4 to 99.6 and reduces stop regret from 0.069 to 0.066. These trends motivate moderate operating points that retain strong development-set performance while bounding candidate processing, matched-control scoring, and offline search.

Training-loss weights. Both loss-weight sweeps favor moderate, nonzero coefficients within the evaluated grids. The selected value $\eta = 0.2$ reaches a quality index of 99.9, compared with 99.4 when the ranking loss is omitted and 99.6 at $\eta = 1$. Likewise, $\zeta = 0.4$ reaches 99.9, compared with 99.0 at $\zeta = 0$ and 99.4 at $\zeta = 1$. The neighboring settings remain competitive—quality is 99.8 at both $\eta = 0.5$

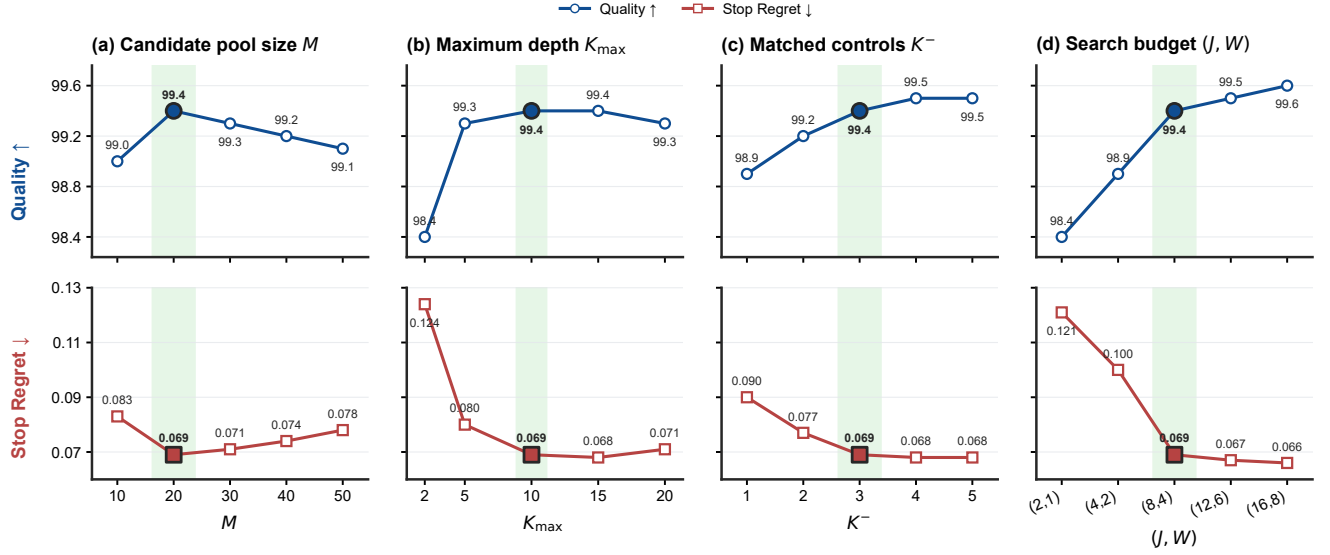


Figure 9: Measured development-set sensitivity of ENOUGH to candidate-pool size M , maximum profile depth $K_{\max}$, number of matched controls K^- , and offline search budget (J, W) . The upper row reports the aggregate quality index (higher is better), and the lower row reports stop regret (lower is better). Shaded bands and black-outlined markers denote the selected settings $M = 20$, $K_{\max} = 10$, $K^- = 3$, and $(J, W) = (8, 4)$.

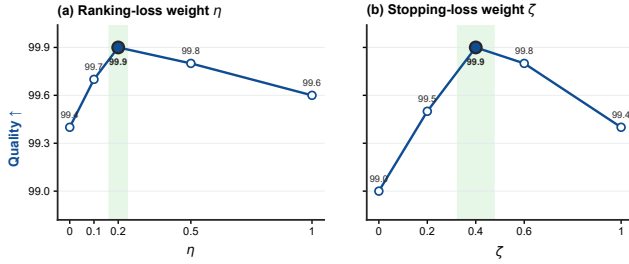


Figure 10: Measured development-set quality sensitivity to the ranking-loss weight η and stopping-loss weight ζ in Eq. (31). Each weight is varied within $[0, 1]$ while the remaining configuration is held fixed. Higher is better; shaded bands and black-outlined markers denote the selected values $\eta = 0.2$ and $\zeta = 0.4$.

and $\zeta = 0.6$ —so the results indicate a favorable region rather than a sharply isolated optimum. The sweeps therefore support balancing the ranking and stopping objectives with value regression. Because Figure 10 reports only aggregate quality, it does not by itself establish how either coefficient affects stopping calibration, profile length, or task-level variability.

E.9 Robustness and Subgroups

Controlled stress tests. Starting from the one-useful-record condition, a near duplicate changes selected length by 0.07 records. The action-conflicting condition changes harm from 12.0% to 19.0% and quality by 0.7 index points (Table 13). The subgroup analysis stratifies history length, match coverage, and input difficulty.

The robustness ladder inserts one perturbation at a time

Candidate ladder	Quality	Tokens	Records	Root STOP	Harm
No useful history	96.3	0	0.00	100.0%	0.0%
+ useful record	99.3	220	0.99	18.2%	12.0%
+ near duplicate	99.3	233	1.06	17.7%	11.9%
+ irrelevant record	99.2	237	1.08	17.0%	13.1%
+ action conflict	98.5	266	1.25	15.0%	19.0%

Table 13: Measured controlled candidate-injection stress test. Each row adds one perturbation to the preceding candidate-pool condition.

Dimension	Bucket	Quality	Tokens	Root STOP	Harm
History	short	98.9	302	26.6%	13.7%
History	medium	99.5	506	16.8%	15.2%
History	long	99.6	727	8.3%	18.0%
Coverage	low	98.5	354	30.1%	18.2%
Coverage	medium	99.3	501	16.9%	15.4%
Coverage	high	99.7	656	8.1%	13.8%
Difficulty	easy	99.5	425	20.2%	11.2%
Difficulty	medium	99.3	514	16.6%	14.9%
Difficulty	hard	98.8	611	13.1%	22.0%

Table 14: Measured subgroup analysis by user-history length, matched-candidate coverage, and input difficulty. Buckets are defined by development-set quantiles and frozen for test evaluation.

into an otherwise controlled pool. Near duplicates test redundancy, random records test distraction, and action conflicts test whether the controller can reject semantically relevant

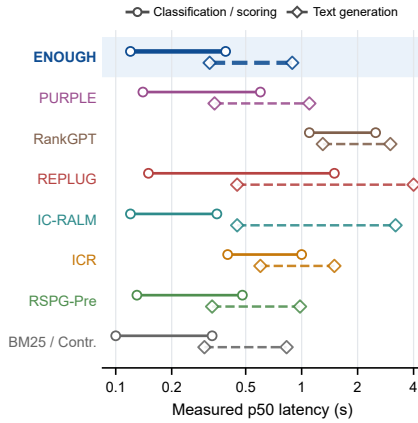
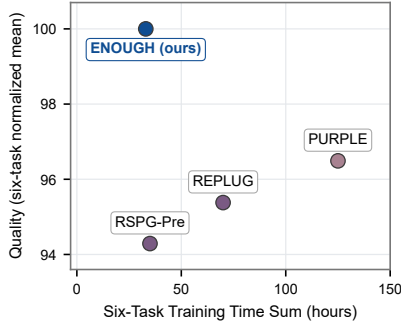


Figure 11: Detailed training-time and latency comparison. Top: Quality versus the measured six-task training-time sum on one A800 for the four methods that require task-specific GPU training; Quality is the equal-weight mean of six task-normalized scores. Bottom: measured batch-size-one p50 latency ranges across classification/scoring tasks (solid lines with circles) and text-generation tasks (dashed lines with diamonds); BM25 and Contriever are grouped in this panel.

but behaviorally inconsistent evidence. Subgroups are defined on development-set quantiles and frozen before test evaluation.

E.10 Online and Offline Efficiency

All online timings use the same single-GPU machine, batch size one, warmed caches, and identical generator settings. We report median and p95 end-to-end latency and decompose the median into retrieval, controller or selector, generation, and residual preprocessing. Multi-call baselines include every reranking or generation call. Offline teacher scoring, cache construction, and additional λ operating points are excluded from online latency but reported explicitly in Table 17.

Training and task-family detail. Figure 11 separates offline training time from task-dependent online latency. ENOUGH reaches Quality 100.00 with a six-task training sum of 33 hours, close to RSPG-Pre at 35 hours and below

Method	Quality	Tokens	Calls	p50 ms	p95 ms
ENOUGH	100.00	510	1	370	440
PURPLE	96.49	905	1	398	474
RankGPT	96.32	869	6	1120	1379
REPLUG	95.38	1388	5	1323	1595
IC-RALM	94.82	1210	7	1463	1793
ICR	94.30	819	4	865	1049
RSPG-Pre	94.29	765	3	796	951
Contriever	92.17	931	1	365	442
BM25	90.96	905	1	360	421

Table 15: Measured online inference cost under identical generation settings. Tokens is the mean selected-profile length; Calls is the number of generator-equivalent invocations; p50/p95 are end-to-end latency in milliseconds.

Method	Retrieval	Controller	Selector	Generator	Other
Contriever-5	17	–	1	362	15
PURPLE-5	18	–	10	357	16
ENOUGH	17	11	–	325	17

Table 16: Measured p50 end-to-end latency decomposition in milliseconds under the common batch-size-one evaluation protocol.

Teacher calls (M)	Cache hit	GPU-hours	Storage (GB)	Extra GPU-h per λ
9.7	74.8%	336	95	95

Table 17: Measured offline teacher-scoring and cache cost for one operating point. Teacher calls are reported in millions.

REPLUG and PURPLE at 70 and 125 hours. In the measured latency comparison, ENOUGH remains in the low-latency group for both task families; BM25/Contriever have slightly lower ranges, whereas RankGPT and the generation paths of REPLUG and IC-RALM extend to substantially higher upper bounds. Because the two panels measure different cost dimensions, we do not collapse them into a single efficiency score.

F Complete Generator Prompt Templates

Figure 12 shows the complete logical system and user messages supplied to the frozen generator before model-specific chat serialization. The gray system instruction and the `<TASK>`, `<USER_HISTORY>`, and `<CURRENT_REQUEST>` delimiters are shared by every method. Within `<CURRENT_REQUEST>`, we retain the official LaMP task instruction verbatim and replace only instance-specific content with descriptive braced fields. The gold block uses our uniform behavioral-record serialization; records are repeated in the order chosen by the selector. If the controller selects `STOP` at the root, the two `<USER_HISTORY>` delimiters remain but enclose no record. Display line breaks inside the blue block are visual wrapping only. The official Qwen3.5-9B and Llama-3.1-8B-

Instruct chat templates subsequently add their model-specific control tokens.

For LaMP-1–5, the historical context and user action correspond to the task fields reported in each figure. Public LaMP-7 histories contain only the user’s prior tweet and no separate pre-action context; accordingly, its `[HIST_CONTEXT]` field is empty and the tweet is serialized as `[USER_ACTION]`. This is a property of the released benchmark schema rather than a task-specific prompt exception.

SYSTEM / Shared instruction (used by panels a-f)

You are solving a LaMP personalization task. Use the user history only when useful. Follow the current request exactly and output only the requested answer.

(a) LaMP-1**Citation Identification****USER / Task tag**

<TASK> LaMP_1 </TASK>

USER / Unified profile serialization

```
<USER_HISTORY>
[TASK] LaMP_1
[HIST_CONTEXT]
{historical_paper_abstract_j}
[USER_ACTION] {historical_paper_title_j}
[META] {"date": {historical_paper_year_j}}

... repeated in ENOUGH-selected order for
j = 1, ..., |S_i| ...
</USER_HISTORY>
```

USER / Official LaMP task instruction

```
<CURRENT_REQUEST>
For an author who has written the paper
with the title "{query_paper_title}",
which reference is related? Just answer
with [1] or [2] without explanation. [1]:
"{candidate_reference_1}" [2]:
"{candidate_reference_2}"
</CURRENT_REQUEST>
```

(b) LaMP-2**Movie Tagging****USER / Task tag**

<TASK> LaMP_2 </TASK>

USER / Unified profile serialization

```
<USER_HISTORY>
[TASK] LaMP_2
[HIST_CONTEXT]
{historical_movie_description_j}
[USER_ACTION] {historical_user_tag_j}
[META] {"date": "{historical_tag_date_j}"}

... repeated in ENOUGH-selected order for
j = 1, ..., |S_i| ...
</USER_HISTORY>
```

USER / Official LaMP task instruction

```
<CURRENT_REQUEST>
Which tag does this movie relate to among
the following tags? Just answer with the
tag name without further explanation.
tags: [sci-fi, based on a book, comedy,
action, twist ending, dystopia, dark
comedy, classic, psychology, fantasy,
romance, thought-provoking, social
commentary, violence, true story]
description: {target_movie_description}
</CURRENT_REQUEST>
```

(c) LaMP-3**Product Rating****USER / Task tag**

<TASK> LaMP_3 </TASK>

USER / Unified profile serialization

```
<USER_HISTORY>
[TASK] LaMP_3
[HIST_CONTEXT] {historical_review_text_j}
[USER_ACTION] {historical_user_score_j}
[META] {"date":
"{historical_review_date_j}"}

... repeated in ENOUGH-selected order for
j = 1, ..., |S_i| ...
</USER_HISTORY>
```

USER / Official LaMP task instruction

```
<CURRENT_REQUEST>
What is the score of the following review
on a scale of 1 to 5? just answer with 1,
2, 3, 4, or 5 without further explanation.
review: {target_review_text}
</CURRENT_REQUEST>
```


(d) LaMP-4

News Headline Generation

USER / Task tag

<TASK> LaMP_4 </TASK>

USER / Unified profile serialization

```
<USER_HISTORY>
[TASK] LaMP_4
[HIST_CONTEXT] {historical_article_text_j}
[USER_ACTION] {historical_user_headline_j}
[META] {"date":
"{historical_article_date_j}"}

... repeated in ENOUGH-selected order for
j = 1, ..., |S_i| ...
</USER_HISTORY>
```

USER / Official LaMP task instruction

```
<CURRENT_REQUEST>
Generate a headline for the following
article: {target_article_text}
</CURRENT_REQUEST>
```

(e) LaMP-5

Scholarly Title Generation

USER / Task tag

<TASK> LaMP_5 </TASK>

USER / Unified profile serialization

```
<USER_HISTORY>
[TASK] LaMP_5
[HIST_CONTEXT]
{historical_paper_abstract_j}
[USER_ACTION] {historical_user_title_j}
[META] {"date": {historical_paper_year_j}}

... repeated in ENOUGH-selected order for
j = 1, ..., |S_i| ...
</USER_HISTORY>
```

USER / Official LaMP task instruction

```
<CURRENT_REQUEST>
Generate a title for the following
abstract of a paper:
{target_paper_abstract}
</CURRENT_REQUEST>
```

(f) LaMP-7

Tweet Paraphrasing

USER / Task tag

<TASK> LaMP_7 </TASK>

USER / Unified profile serialization

```
<USER_HISTORY>
[TASK] LaMP_7
[HIST_CONTEXT]
[USER_ACTION] {historical_user_tweet_j}
[META] {"date":
"{historical_tweet_date_j}"}

... repeated in ENOUGH-selected order for
j = 1, ..., |S_i| ...
</USER_HISTORY>
```

USER / Official LaMP task instruction

```
<CURRENT_REQUEST>
Paraphrase the following tweet without any
explanation before or after it:
{target_tweet_description}
</CURRENT_REQUEST>
```

Figure 12: Complete logical generator-prompt templates for (a) LaMP-1 personalized citation identification, (b) LaMP-2 personalized movie tagging, (c) LaMP-3 personalized product rating, (d) LaMP-4 personalized news headline generation, (e) LaMP-5 personalized scholarly title generation, and (f) LaMP-7 personalized tweet paraphrasing. The system instruction shown on the first page is shared by all six templates.